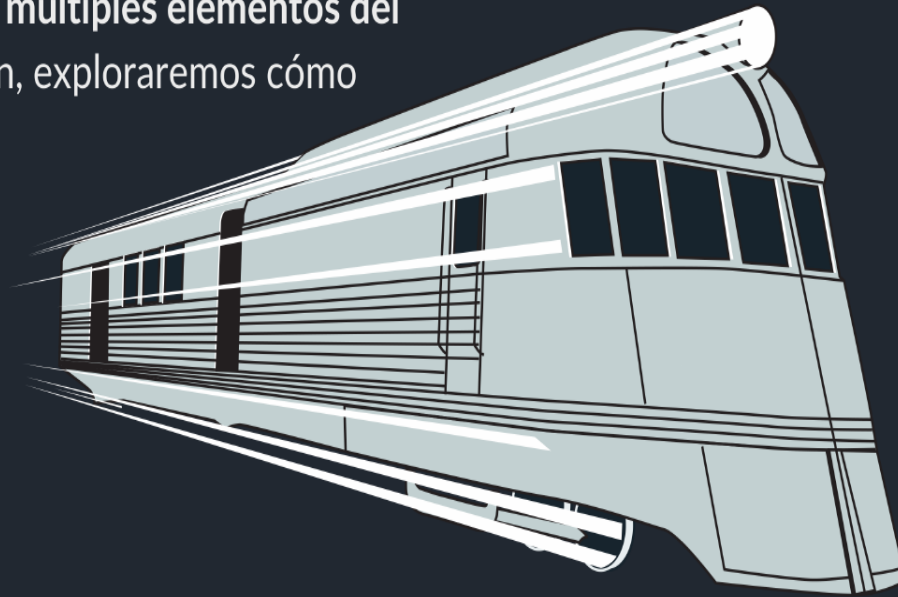


Introducción a los arreglos en c++

Unidimensionales

Los arreglos son una **estructura de datos** fundamental en C++ que nos **permite almacenar múltiples elementos del mismo tipo**. En esta presentación, exploraremos cómo trabajar con arreglos en C++



Introducción a los arreglos en c++

Unidimensionales

Imagina un tren, con dos limitantes muy importantes, 1)
Solo puede llevar un mismo tipo de objeto/animal, 2) Solo
puede llevar un objeto/animal por vagón.



Introducción a los arreglos en c++

Unidimensionales

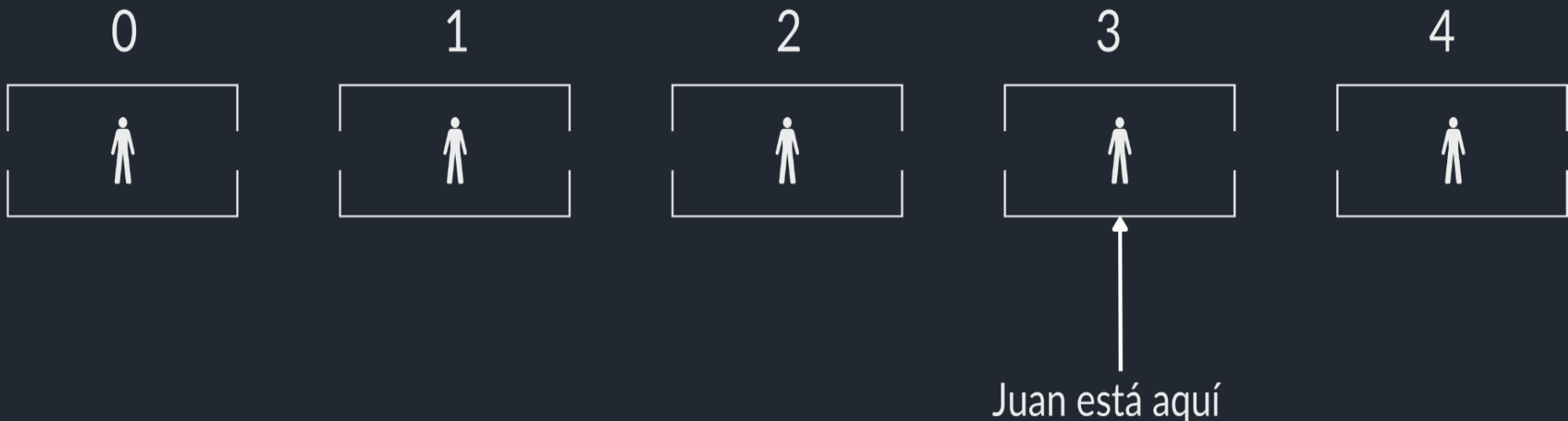
Además, cada vagón tiene un número asignado (partiendo de 0), para que sea fácil señalarlos. Llamémoslo “índice”.



Introducción a los arreglos en c++

Unidimensionales

Ahora, asumamos que Juan se encuentra en el vagón enumerado con el "3", si nuestro tren se llama "Titanic", podríamos decir que: Juan se encuentra, en el tercer vagón del Titanic.



Declaración de Arreglos

Declaración de Arreglos

Un arreglo es exactamente lo mismo, es una colección de elementos del mismo tipo (como, las **personas del tren**), y se puede acceder a los elementos por medio de índices (como los números que asignamos a los vagones).



```
// Se declara...  
char letras[5];  
// Se declara, y se asigna  
char letras[] = {'a', 'm', 'n', 'r', 'h'};
```

0



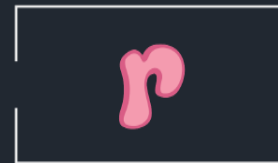
1



2



3



4



Declaración de Arreglos

Declaración de Arreglos

Los arreglos pueden hacer uso de cualquier **tipo** (números, caracteres, cadenas...) en c++, ya sea un tipo primitivo, o uno personalizado.



```
// Puede ser un caracter, o un string, o...  
char|int|std::string|etc... letras[5];
```

0

1

2

3

4

Declaración de Arreglos

Declaración de Arreglos

Si por ejemplo quiero acceder a la “n” dentro de mi arreglo, puede indicar la posición, de la misma manera en que señalaba los vagones.



```
char letras[] = {'a', 'm', 'n', 'r', 'h'};  
std::cout << letras[2]; // n
```

0



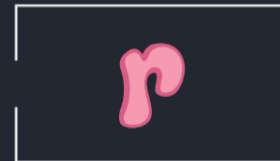
1



2



3



4



La “n” está aquí

Práctica Guiada

Recorrido de arreglos

La palabra “recorrer” es solo una forma de decir que vamos a pasar por cada uno de los elementos del arreglo. Partiendo desde un punto “n” hasta un punto “k”. Para lograrlo podemos usar cualquier estructura repetitiva de nuestra preferencia.

KEYWORDS: Recorrer, iterar, barrer, por cada...

0



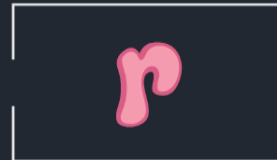
1



2



3



4



Tamaño de Arreglos

SizeOf

La palabra reservada “**sizeof**”, nos indica el tamaño de un elemento. Con un poco de lógica se puede determinar el tamaño de un arreglo.

```
char letras[] = {'a', 'm', 'n', 'r', 'h'};  
std::cout << letras[2]; // n  
  
sizeof(letras) / sizeof(letras[0]); // 5?
```

0



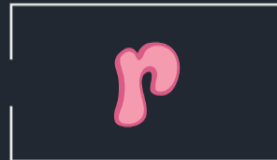
1



2



3



4



Problema común

Desbordamiento

Cuidado con acceder a elementos fuera del rango del arreglo.



```
char letras[] = {'a', 'm', 'n', 'r', 'h'};  
// Este arreglo solo llega hasta el indice 4  
std::cout << letras[5];
```

0



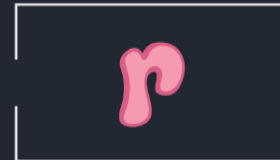
1



2



3



4



Ejercicios - Recorrido y búsqueda

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones necesarias.

Ejercicio 1 - Suma de elementos

Desarrolle un programa en C++ que calcule la suma de todos los elementos de un arreglo de enteros. El tamaño del arreglo y sus elementos deben ser ingresados por el usuario.

Ejercicio 2 - Buscar un elemento

Desarrolle un programa en C++ que permita al usuario buscar un elemento específico en un arreglo de enteros. El programa debe indicar si el elemento se encuentra en el arreglo o no.

Ejercicio 10.2 - Calcular la media

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite al usuario 5 números y almacénelos en un arreglo.
- Seguidamente, recorra el arreglo y calcule la media de los números almacenados.
- El cálculo debe realizarse mediante una función que reciba el arreglo y retorne la media calculada.
- Ejemplo: `int vector[] = {13, 21, 45, 5, 98}` -> Media = 36.4

Ejercicio 10.4 - Número menor

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Cree una función en C++ que reciba un arreglo de números y su tamaño como parámetros.
- Recorra el arreglo y determine cuál es el número menor.
- La función deberá retornar el valor menor encontrado.
- Ejemplo: {13, 21, 45, 5, 98} -> Valor a retornar: 5

Ejercicio 10.5 - Número mayor

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Cree una función en C++ que reciba un arreglo de números y su tamaño como parámetros.
- Recorra el arreglo y determine cuál es el número mayor.
- La función deberá retornar el valor mayor encontrado.
- Ejemplo: {13, 21, 45, 5, 98} -> Valor a retornar: 98

Ejercicio 10.6 - Positivos, negativos y ceros

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite 10 números enteros y almacénelos en un arreglo.
- Recorra el arreglo y determine cuántos valores son positivos, cuántos son negativos y cuántos son iguales a cero.
- Además, calcule la suma total de los elementos.

Ejercicio 10.7 - Pares, impares y múltiplos

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite 12 números enteros y almacénelos en un arreglo.
- Determine cuántos son pares, cuántos son impares, cuántos son múltiplos de 3 y cuántos son múltiplos de 5.
- Un mismo número puede pertenecer a más de una clasificación.

Ejercicio 10.8 - Buscar todas las apariciones

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite los elementos de un arreglo y posteriormente un número que desea buscar.
- Recorra todo el arreglo y muestre cada posición en la que aparece el valor.
- No finalice la búsqueda al encontrar la primera coincidencia.
- Al terminar, indique cuántas veces apareció el número. Si no apareció, muestre un mensaje.

Ejercicio 10.9 - Segundo número mayor

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite 10 números enteros diferentes y almacénelos en un arreglo.
- Determine cuál es el número mayor y cuál es el segundo número mayor.
- No ordene el arreglo.

Ejercicio 10.10 - Invertir un arreglo

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite 8 números enteros y almacénelos en un arreglo.
- Muestre primero los elementos en el orden en que fueron ingresados.
- Después muéstrellos desde la última posición hasta la primera.
- Reto adicional: intercambie realmente los elementos para que el arreglo quede invertido.

Ejercicio 10.11 - ¿Está ordenado?

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite 10 números enteros.
- Determine si los elementos están ordenados de menor a mayor.
- No debe ordenar el arreglo: únicamente debe analizarlo.
- Compare cada elemento con el que se encuentra inmediatamente después.

Ejercicio 10.12 - Valores repetidos

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite 10 números enteros y almacénelos en un arreglo.
- Determine cuáles valores se repiten y cuántas veces aparece cada uno.
- Evite imprimir varias veces el mismo valor repetido.
- Compare los elementos del arreglo entre sí.

Ejercicio 10.13 - Eliminar un elemento

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite 10 números enteros y luego un valor que se desea eliminar.
- Busque la primera aparición del valor.
- Si existe, desplace una posición hacia la izquierda todos los elementos posteriores para cubrir el espacio.
- Muestre el arreglo resultante.

Ejercicio 10.14 - Rotación a la derecha

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite 8 números enteros.
- Desplace todos los elementos una posición hacia la derecha.
- El último elemento debe pasar a ocupar la posición 0.
- Ejemplo: {1, 2, 3, 4} -> {4, 1, 2, 3}.

Ejercicio 10.15 - Mayor diferencia

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite 10 números enteros.
- Calcule la diferencia absoluta entre cada par de elementos consecutivos.
- Determine cuál fue la mayor diferencia.
- Además, indique entre cuáles dos posiciones del arreglo ocurrió.

Ejercicio 10.16 - Reto: racha creciente

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite 12 números enteros y almacénelos en un arreglo.
- Determine la longitud de la secuencia consecutiva creciente más larga.
- Ejemplo: {8, 2, 4, 7, 9, 3} contiene una racha creciente de 4 elementos: 2, 4, 7, 9.
- No ordene el arreglo.

Ejercicio 10.17 - Reto: comparar mitades

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite un arreglo de 10 números enteros.
- Calcule la suma de los primeros 5 elementos y la suma de los últimos 5.
- Indique cuál mitad tiene la suma mayor o si ambas sumas son iguales.
- Después determine en cuál mitad se encuentra el elemento mayor de todo el arreglo.

Ejercicio 10.18 - Reto: vecino mayor

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Solicite 10 números enteros.
- Para cada posición interna del arreglo, determine si su valor es mayor que el elemento anterior y que el siguiente.
- Muestre las posiciones que cumplen esa condición.
- Ejemplo: en {2, 8, 3}, el 8 es mayor que sus dos vecinos.

Ejercicio 10.19 - Elementos que no están en otro vector

ANÁLISIS ANTES DE PROGRAMAR

Identifique las variables, el tamaño del arreglo, los índices, los ciclos y las condiciones que necesita.

- Trabaje con dos objetos que contengan un vector de números.
- Escriba un método que reciba como parámetro una instancia obj2.
- El método deberá recorrer el vector del objeto actual y mostrar únicamente los elementos que no se encuentran en el vector de obj2.
- Por ejemplo, si al comparar ambos vectores los únicos valores del primero que no aparecen en obj2 son 6 y 5, el método deberá imprimir: 6 y 5.
- Analice cómo comprobar, para cada elemento del primer vector, si existe o no dentro del segundo vector.